

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 8

Виконала:
студентка групи КН-25-1
Марусіченко А. О.
Перевірив:
доцент кафедри КІЕ
Сидоренко В. М.

Кременчук 2026

Тема: Жадібні алгоритми. Наближене розв’язання екстремальних задач

Мета: набути практичних навичок застосування деяких жадібних алгоритмів для розв’язання екстремальних задач.

Порядок виконання роботи

1. Розв’язати задачу комівояжера для зваженого графа: [(1,3,7), (1,4,9), (1,5,11), (2,3,6), (2,4,8), (2,5,10)], використовуючи код.

Код для виконання завдання:

```
from itertools import permutations
import networkx as nx
G = nx.Graph()
G.add_nodes_from(range(1, 6))
G.add_weighted_edges_from([
    (1, 3, 7), (1, 4, 9), (1, 5, 11),
    (2, 3, 6), (2, 4, 8), (2, 5, 10)
])
INFINITY = pow(10, 20)
def get_path_length(path : (_getitem_, _len_)): 1 usage new *
    path_length = 0
    for i, v1 in enumerate(path):
        v2 = path[(i + 1) % len(path)]
        if not G.has_edge(v1, v2):
            return INFINITY
        path_length += G[v1][v2]["weight"]
    return path_length

def node_permutations(G : (nodes), init_node_index): 1 usage new *
    nodes = list(G.nodes())
    init_node = nodes[init_node_index]
    nodes.remove(init_node)
    return [[init_node] + list(a_tuple) for a_tuple in permutations(nodes)]

def TSP_BruteForce(G : (nodes), init_node_index): 1 usage new *
    min_path = None
    min_path_length = INFINITY
    for path in node_permutations(G, init_node_index):
        path_length = get_path_length(path)
        if path_length < min_path_length:
            min_path, min_path_length = path, path_length
    return min_path, min_path_length

best_route, total_cost = TSP_BruteForce(G, init_node_index=0)

print("=" * 40)
if total_cost == INFINITY:
    print("Замкнений маршрут (гамільтонів цикл) не існує для даного графа!")
else:
    print(f"Найкоротший маршрут: {' -> '.join(map(str, best_route))} -> {best_route[0]}")
    print(f"Мінімальна вартість: {total_cost}")
print("=" * 40)
```

Рисунок 1 – Код для знаходження найкоротшого шляху

```

C:\Users\Anna\PycharmProjects\matstat_Marusichenko\.venv\Scripts\p
=====
Замкнений маршрут (гамільтонів цикл) не існує для даного графа!
=====
Process finished with exit code 0

```

Рисунок 2 – Результат

2. Візуалізувати граф.

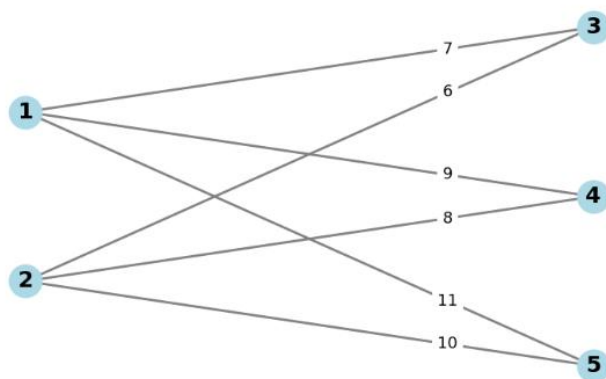


Рисунок 3 – Граф

3. Обґрунтувати асимптотику для обох алгоритмів.

Алгоритм грубої сили $O(n!)$.

Цей метод просто перебирає всі можливі варіанти маршруту. Програма бере першу вершину, крутить усі інші комбінації між собою і для кожної рахує загальну довжину, щоб знайти найкращу. Варіантів стає так багато з кожною новою точкою, що метод підходить лише для маленьких графів, інакше комп'ютер просто зависне.

Алгоритм найближчого сусіда $O(n^2 \log n)$.

Тут логіка жадібна — ми дивимося тільки на те, що поруч. Програма стоїть у вершині, перевіряє сусідні точки, вибирає найкоротшу дорогу за допомогою швидкого сортування і йде туди. Так ми робимо для кожної точки графа. Рахується все миттєво, але через такий поспіх фінальний шлях не завжди виходить ідеальним.

Висновки: набула практичних навичок застосування деяких жадібних алгоритмів для розв'язання екстремальних задач.

Контрольні запитання

1. Жадібний алгоритм – Це підхід, який вирішує задачу покроково і на кожному етапі вибирає найвигідніший варіант прямо зараз. Він не прораховує майбутні кроки, а просто бере найкраще рішення в поточний момент.

2. Принципи роботи: Велика задача ділиться на послідовні кроки, на кожному з яких приймається локально найкраще рішення. При цьому діє принцип незворотності: зроблений вибір ніколи не скасовується і не перераховується заново.

3. Жадібний алгоритм діє наосліп, робить вибір один раз і тому працює швидко, але може помилитися. Динамічне програмування прораховує всі варіанти наперед, розбиваючи задачу на підзадачі, тому воно набагато повільніше, але гарантує 100% точний результат.

4. Сюди відноситься дробова задача про рюкзак, алгоритм Дейкстри для пошуку найкоротшого шляху, алгоритми Пріма і Краскала для побудови мінімальних мереж, а також звичайна видача решти в автоматах найбільшими купюрами.

5. Вони не гарантують ідеальний загальний результат. Якщо задача складна (наприклад, речі в рюкзак можна класти тільки цілими), жадібний вибір на перших кроках може швидко загнати програму в глухий кут і видати найгірший фінальний варіант.

6. Через їхню миттєву швидкість. Коли точний перебір змушує комп'ютер зависнути через мільярди варіантів, жадібний метод видає цілком адекватну відповідь за частку секунди, чого для реального життя (як-от для навігатора в телефоні) зазвичай більш ніж достатньо.